Dataset annotation

Dataset annotation
1. Dataset collection
    1.1. Extract the images
    1.2. Dataset path
2. The use of Label Studio
    2.1. Installing Label Studio
    2.2. Starting Label Studio
        2.2.1. The permission of the shared folder
        2.2.2. Starting Label Studio
        2.2.3. Accessing Label Studio
            The host machine
            The same LAN
            The first use
3. Dataset annotation
    3.1. Creating a project
    3.2. The project name
    3.3. Importing the training set
    3.4. The label settings
    3.5. Dataset annotation
    3.6. Exporting the dataset
    3.7. The directory structure of the dataset
    3.8. The dataset configuration file
References

To recognize a specific object or to improve the recognition accuracy of an object, the user needs to train the model himself, and the dataset collection and annotation are an indispensable part of this.

1. Dataset collection

Description: for the recognition of a specific scene, the user can use a camera to record a video of the object from different angles, and then cut the images out of the video as the training set

1.1. Extract the images

Here is a simple example: extract one image frame every 15 frames from the video as the dataset

```
import cv2
import os

# Path to the input video file
video_file =
'/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_video/orange.mkv'

# Path to the output folder to save the frames
output_folder = '/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_pic'

# Check if the output folder exists, if not, create it
if not os.path.exists(output_folder):
```

```
    os.makedirs(output_folder)

# Open the video file using OpenCV
cap = cv2.VideoCapture(video_file)

# Get the total number of frames in the video
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Get the frames per second (fps) of the video
fps = cap.get(cv2.CAP_PROP_FPS)

# Frame counter to keep track of the current frame number
frame_number = 0

# Define the interval for extracting frames (every 15th frame)
frame_interval = 15

# Initialize a new counter for saved frames (starting from 1)
saved_frame_number = 1

while True:
    ret, frame = cap.read()

    # If the frame is not successfully read, exit the loop
    if not ret:
        break

    # Save every 15th frame
    if frame_number % frame_interval == 0:
        # Format the frame number starting from 1 (e.g., 1.png, 2.png)
        image_name = f'{saved_frame_number}.png'
        image_path = os.path.join(output_folder, image_name)

        # Save the current frame as a PNG image
        cv2.imwrite(image_path, frame)
        print(f'Saving frame {frame_number}/{total_frames} as {image_name}')

        # Increment the saved frame counter
        saved_frame_number += 1

    # Increment the frame counter for the video
    frame_number += 1

# Release the video capture object
cap.release()

# Print message when processing is complete
print("Video processing complete! Every 15th frame saved as an image!")
```

1.2. Dataset path

Sample video: /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_video

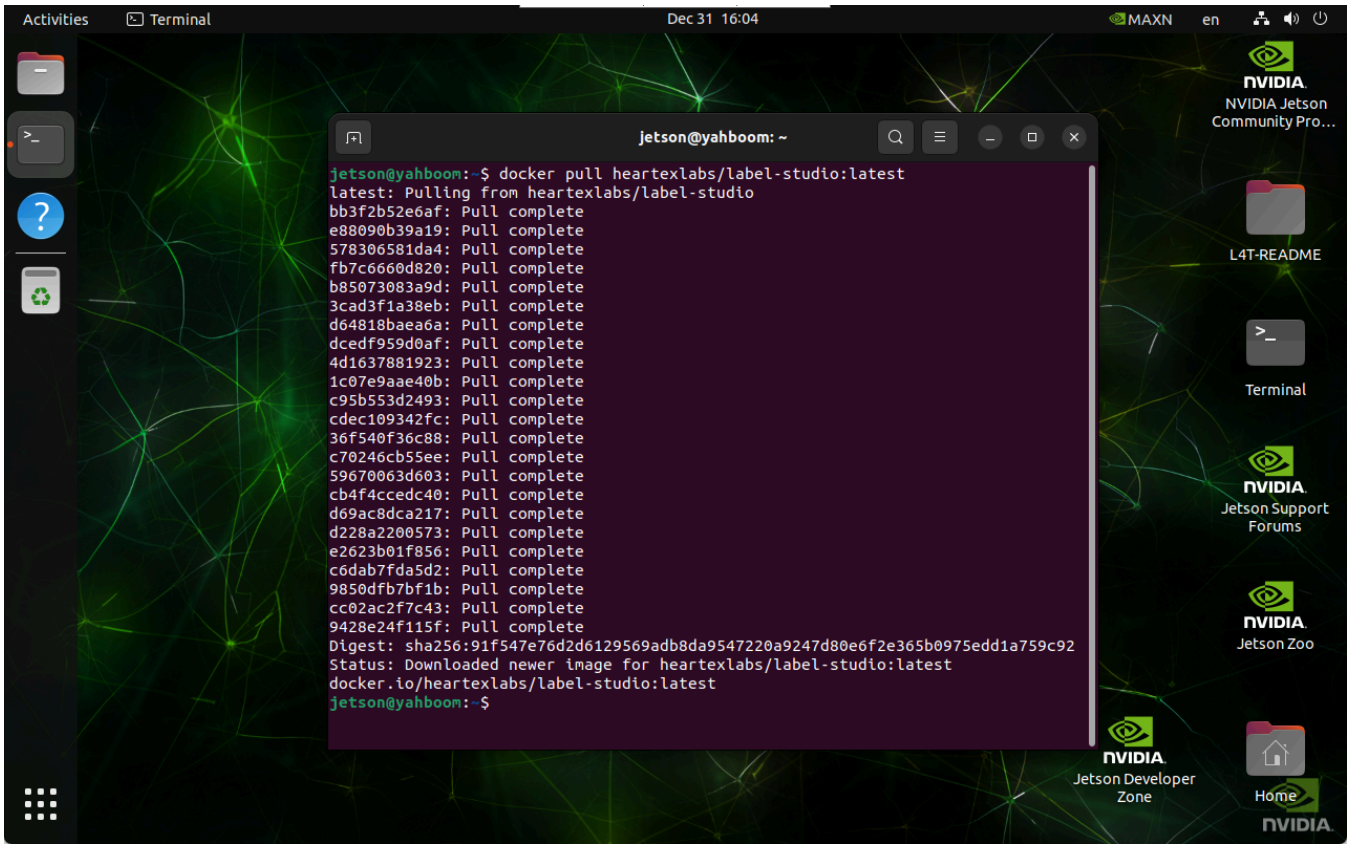
Sample images: /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_pic

2. The use of Label Studio

Label Studio is an open-source data annotation platform, used for data annotation and for the management of the annotation tasks, and it supports various types of data input and annotation formats.

2.1. Installing Label Studio

```
docker pull heartexlabs/label-studio:latest
```



2.2. Starting Label Studio

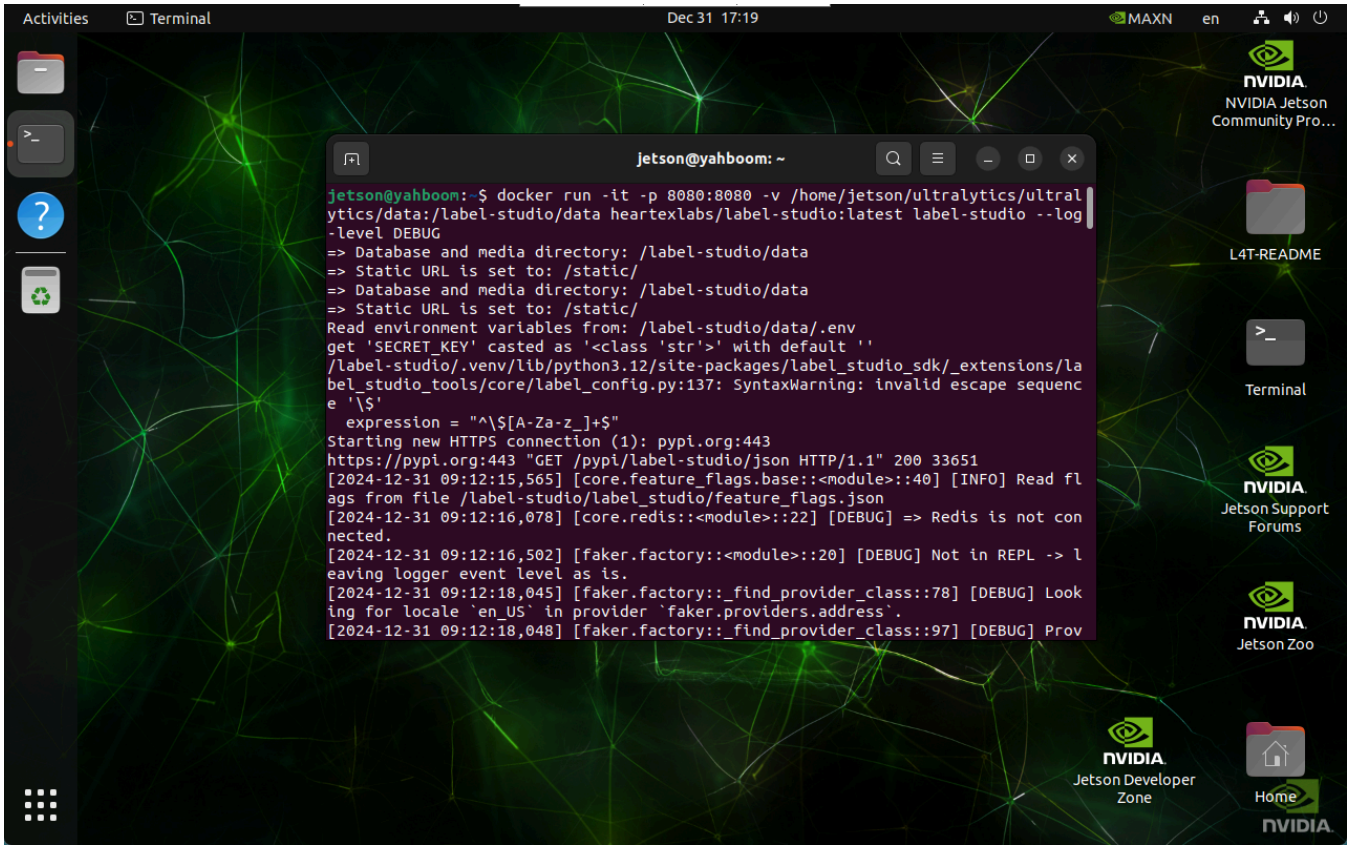
2.2.1. The permission of the shared folder

Share the dataset that has been prepared in advance to the corresponding folder of Label Studio:

```
sudo chmod 777 /home/jetson/ultralytics/ultralytics/data/
```

2.2.2. Starting Label Studio

```
sudo docker run -it -p 8080:8080 -v /home/jetson/ultralytics/ultralytics/data:/label-studio/data heartexlabs/label-studio:latest label-studio --log-level DEBUG
```



Note:

If the port 8080 is already occupied, please change the port mapping function and replace the first 8080 of the -p parameter with 8081 or another suitable port.

```
sudo docker run -it -p 8081:8080 -v /home/jetson/ultralytics/ultralytics/data:/label-studio/data heartexlabs/label-studio:latest label-studio --log-level DEBUG
```

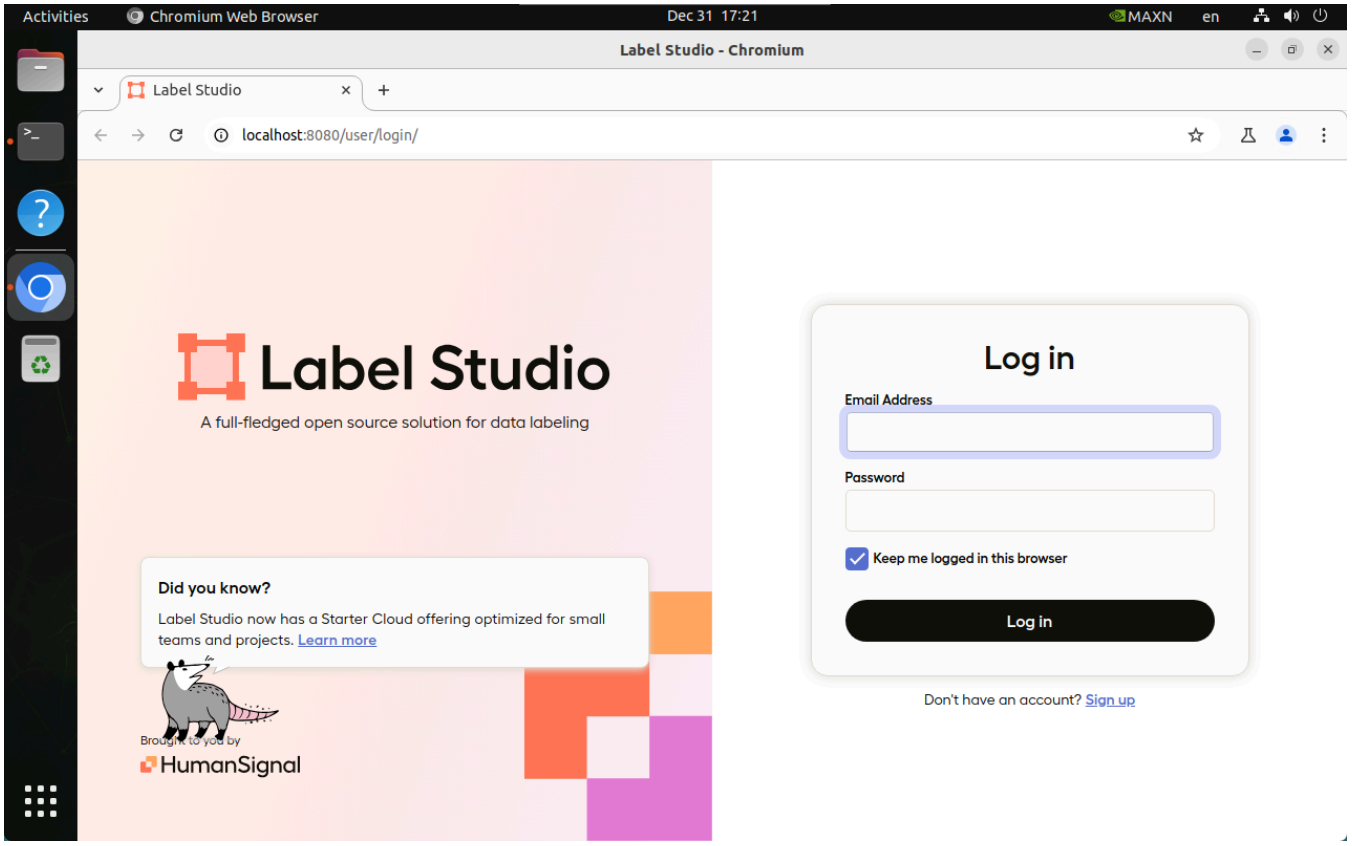
After the port has been changed, Label Studio can be accessed at the new port address in the browser, such as http://localhost:8081/

2.2.3. Accessing Label Studio

The host machine

The access method for the Jetson series development boards; on the first entry you need to enter the user name and the password yourself

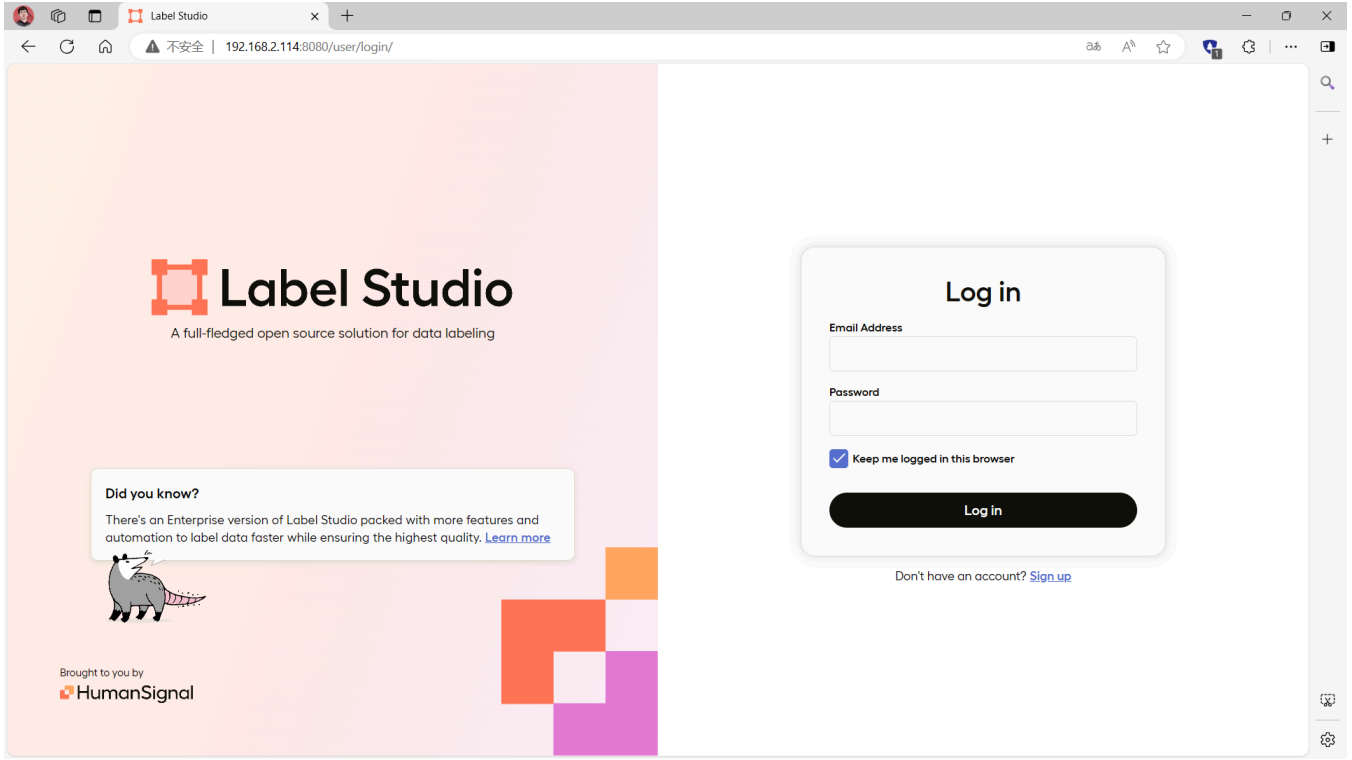
```
http://localhost:8080/
```



The same LAN

A device in the same LAN can access it according to the IP of the mainboard:8080

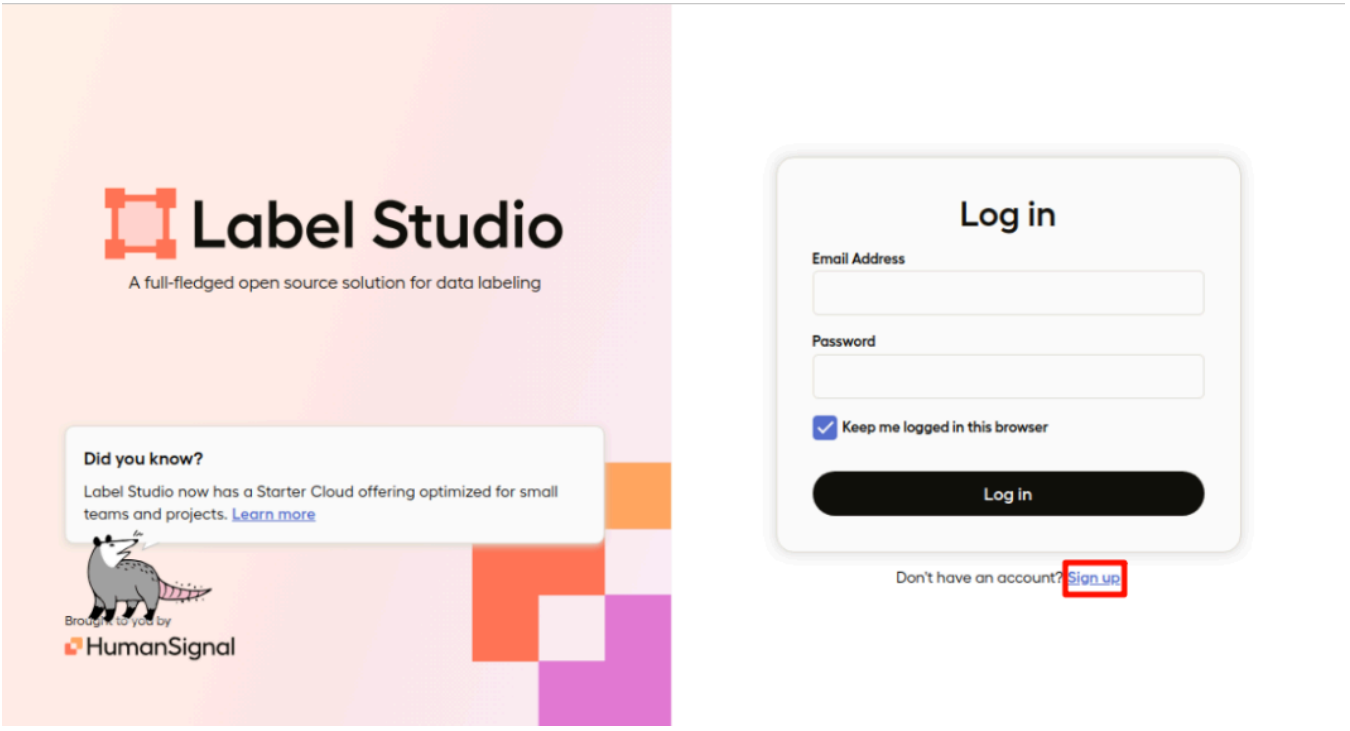
```
http://mainboard IP:8080/ # For example: http://192.168.2.114:8080/
```

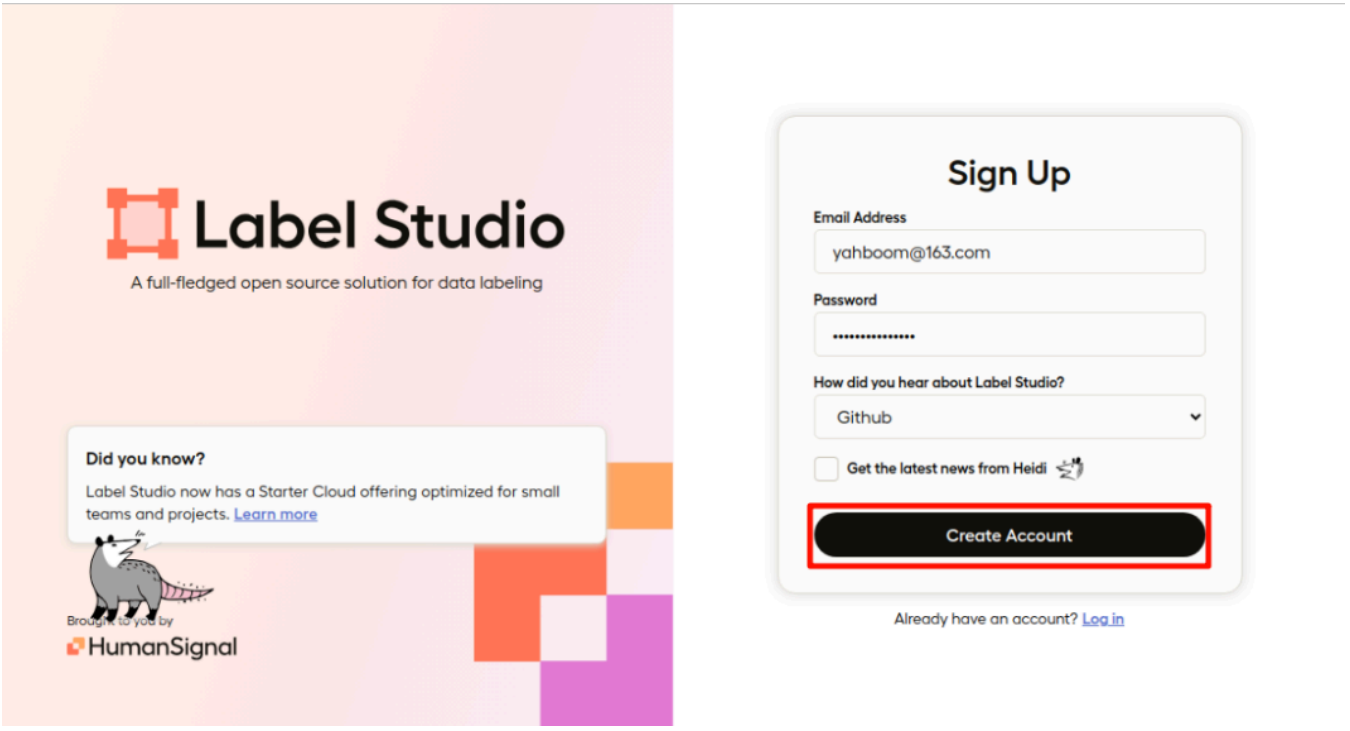


The first use

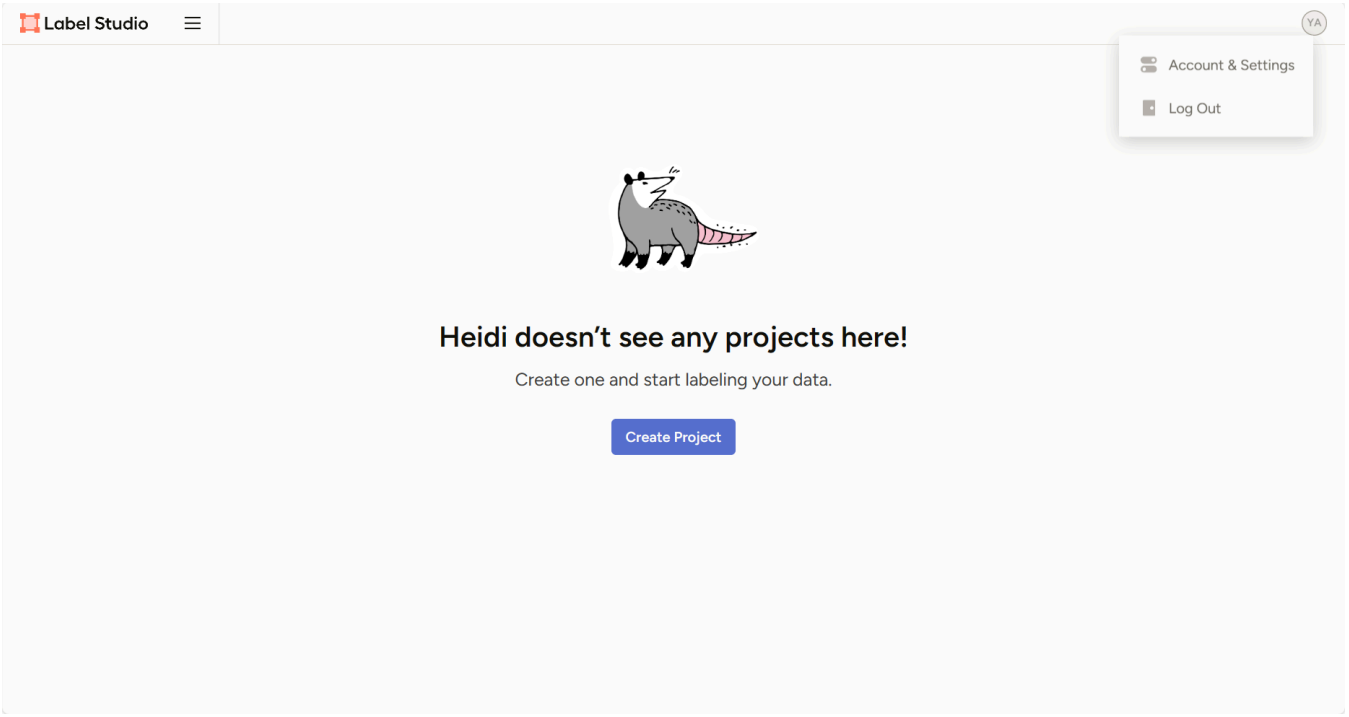
The first use requires you to register and log in with an account yourself; the factory account information is as follows (the account information is a fictitious one):

```
Account: yahboom@163.com
Password: yahboom@163.com
```





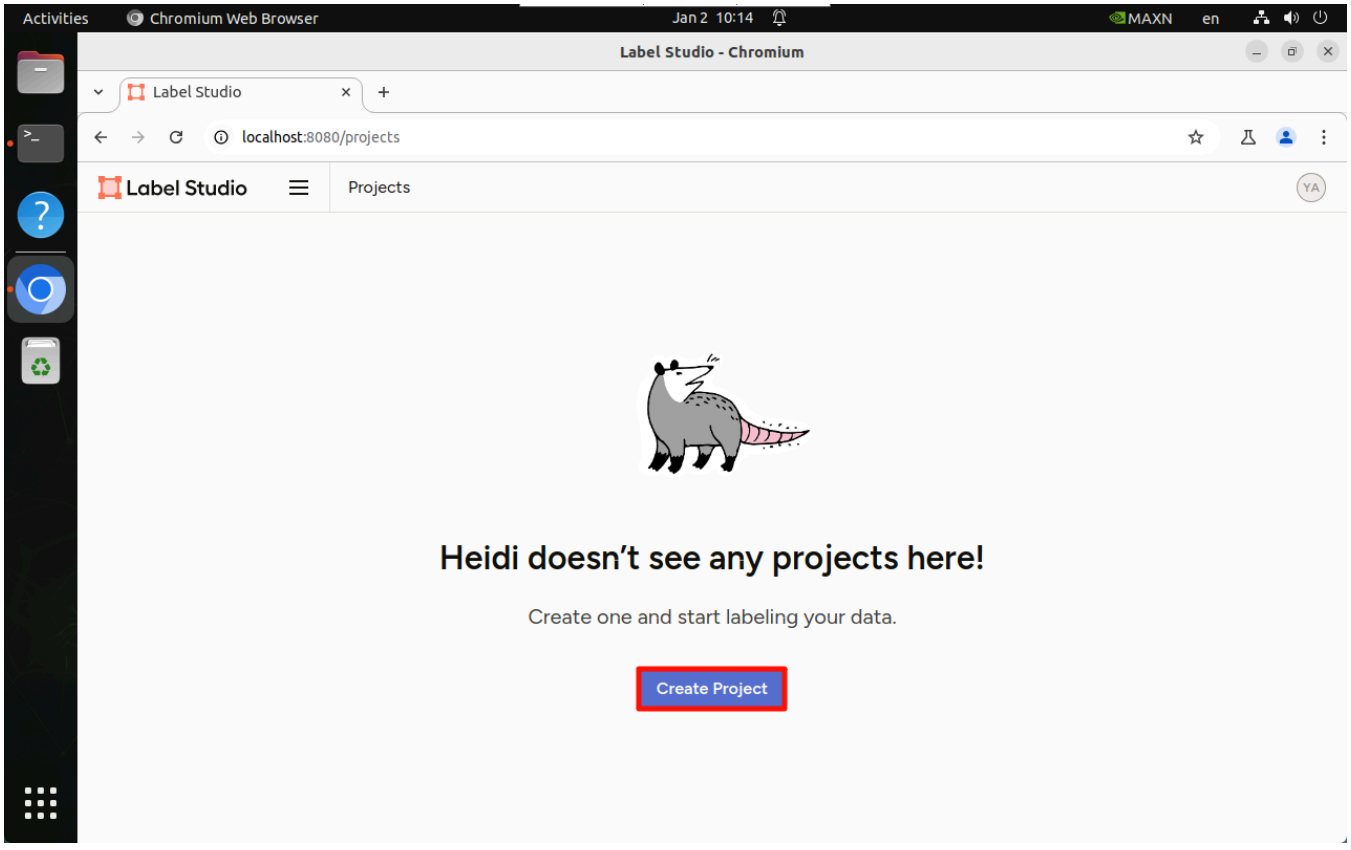
After the registration is complete, the web address logs in automatically:



3. Dataset annotation

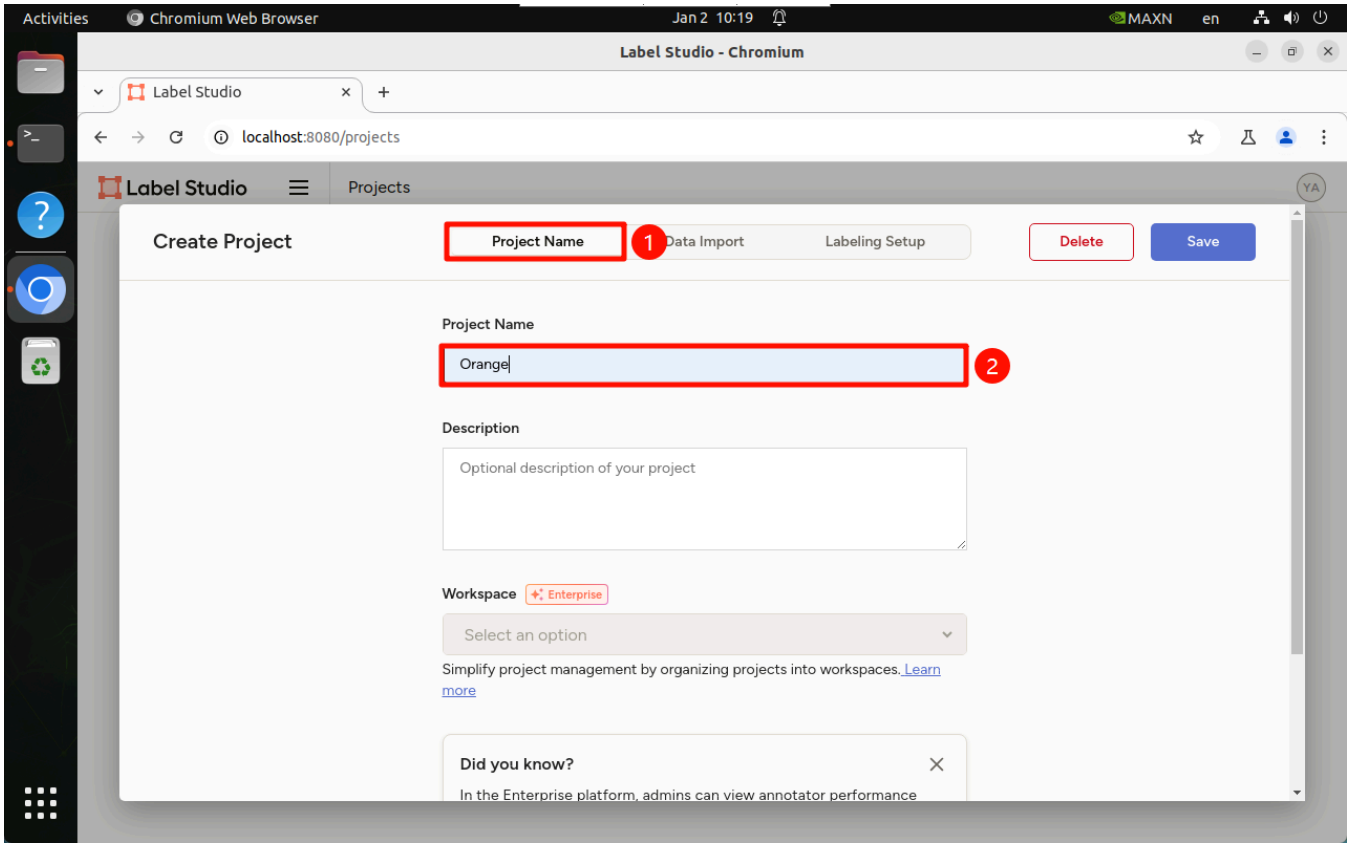
The demo dataset is on the mainboard, so access Label Studio on the mainboard.

3.1. Creating a project



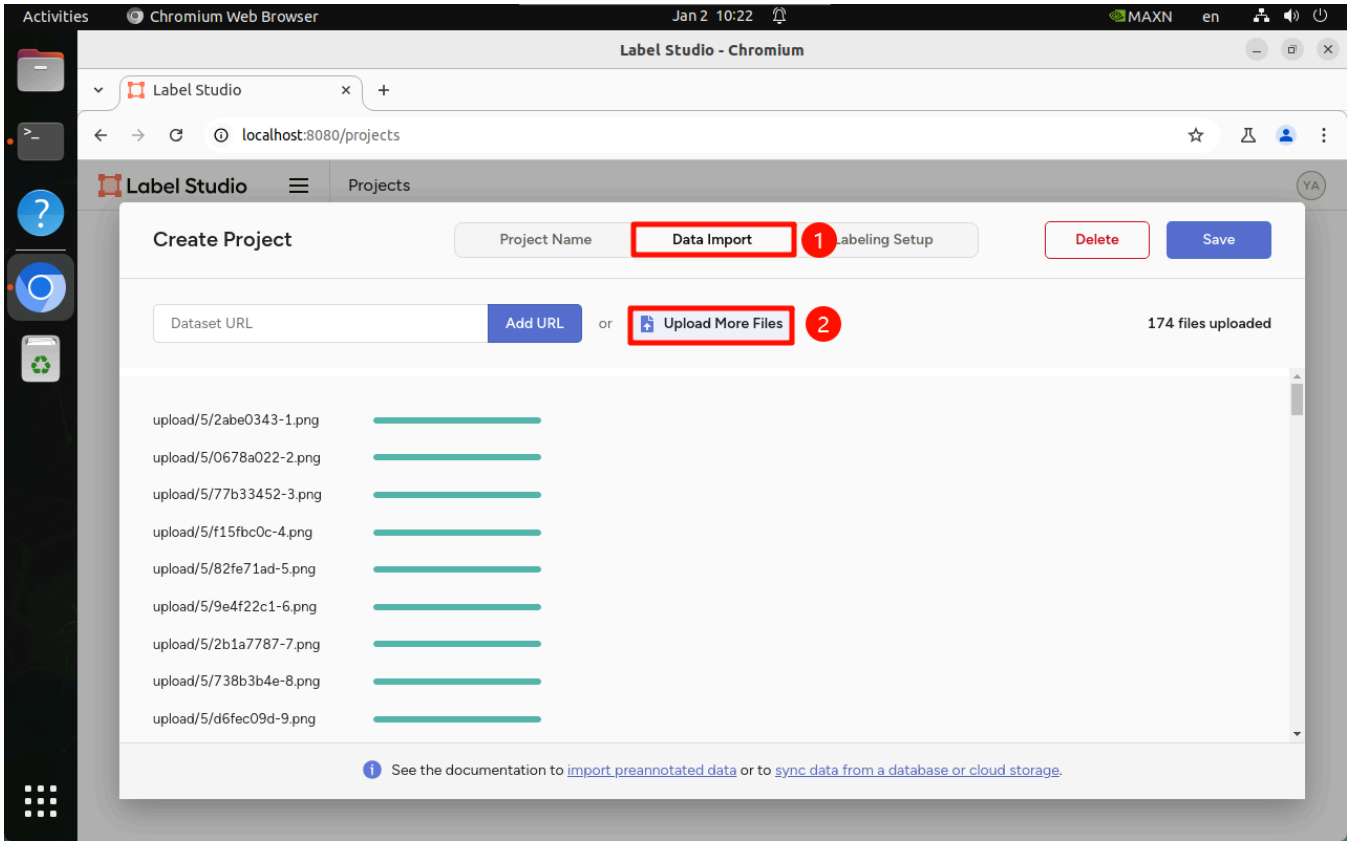
3.2. The project name

The project name can be named freely; name it according to your own training set:



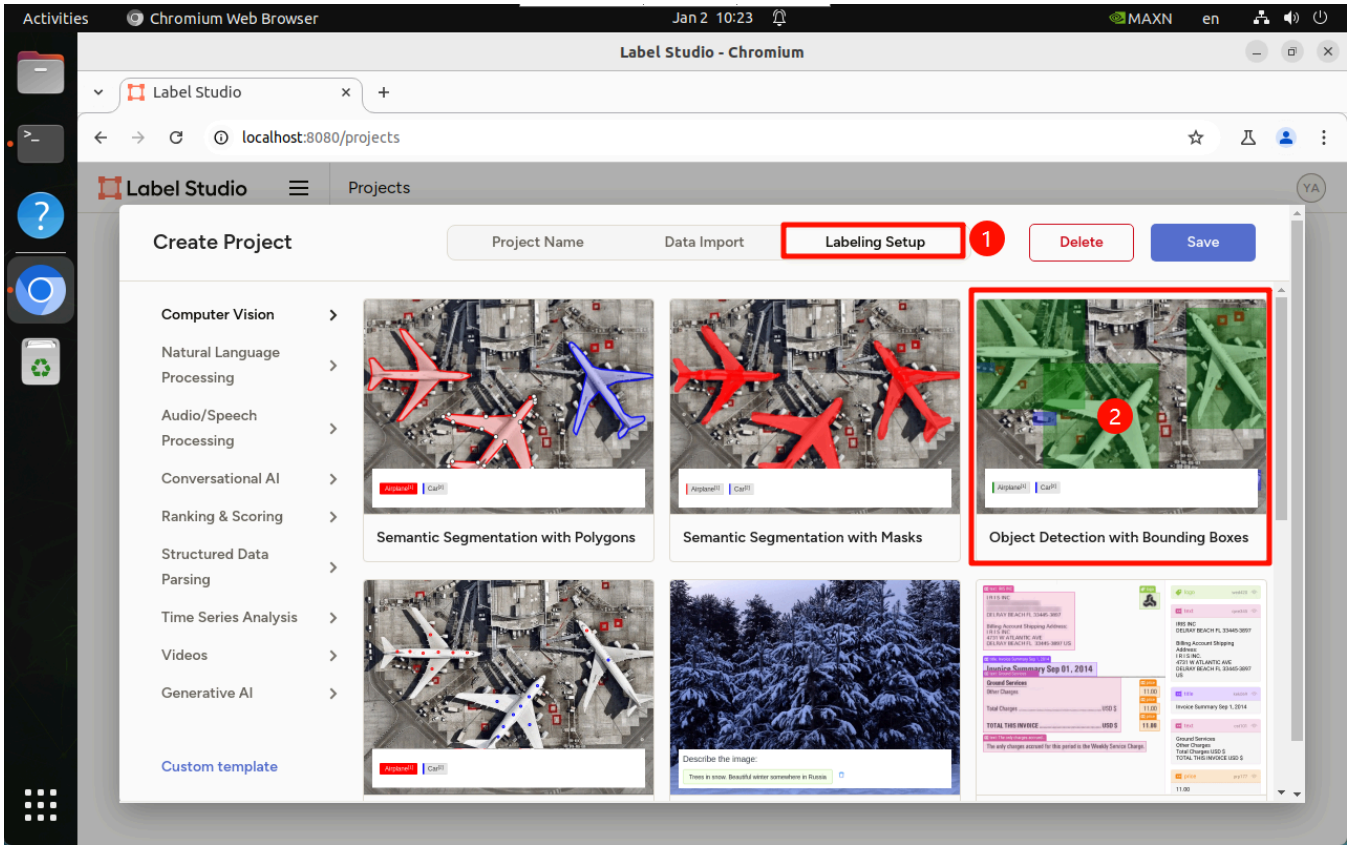
3.3. Importing the training set

Manually import the dataset that has been prepared in advance: too many images cannot be imported at a time, so it is recommended to select 50-100 images each time and then import them several times

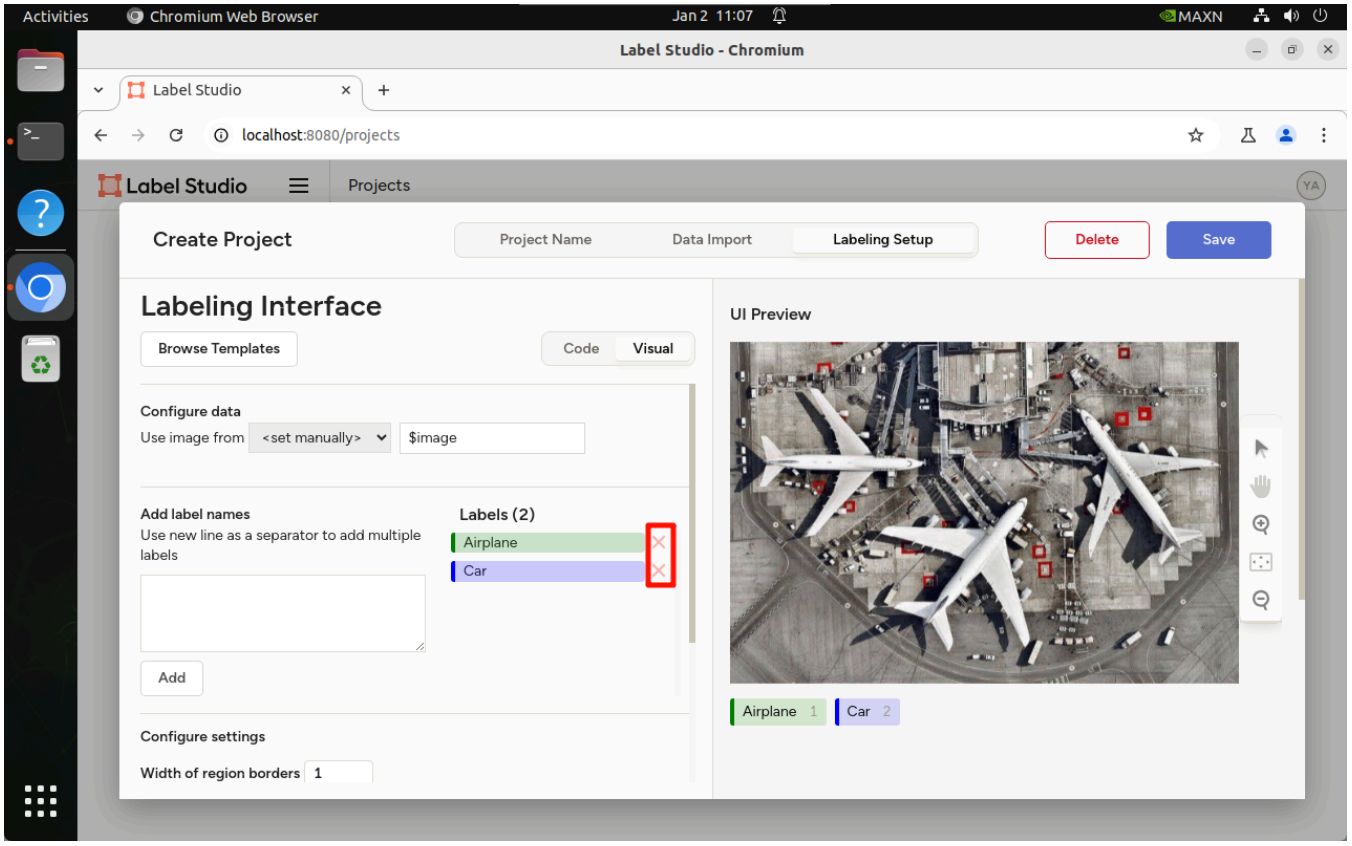


3.4. The label settings

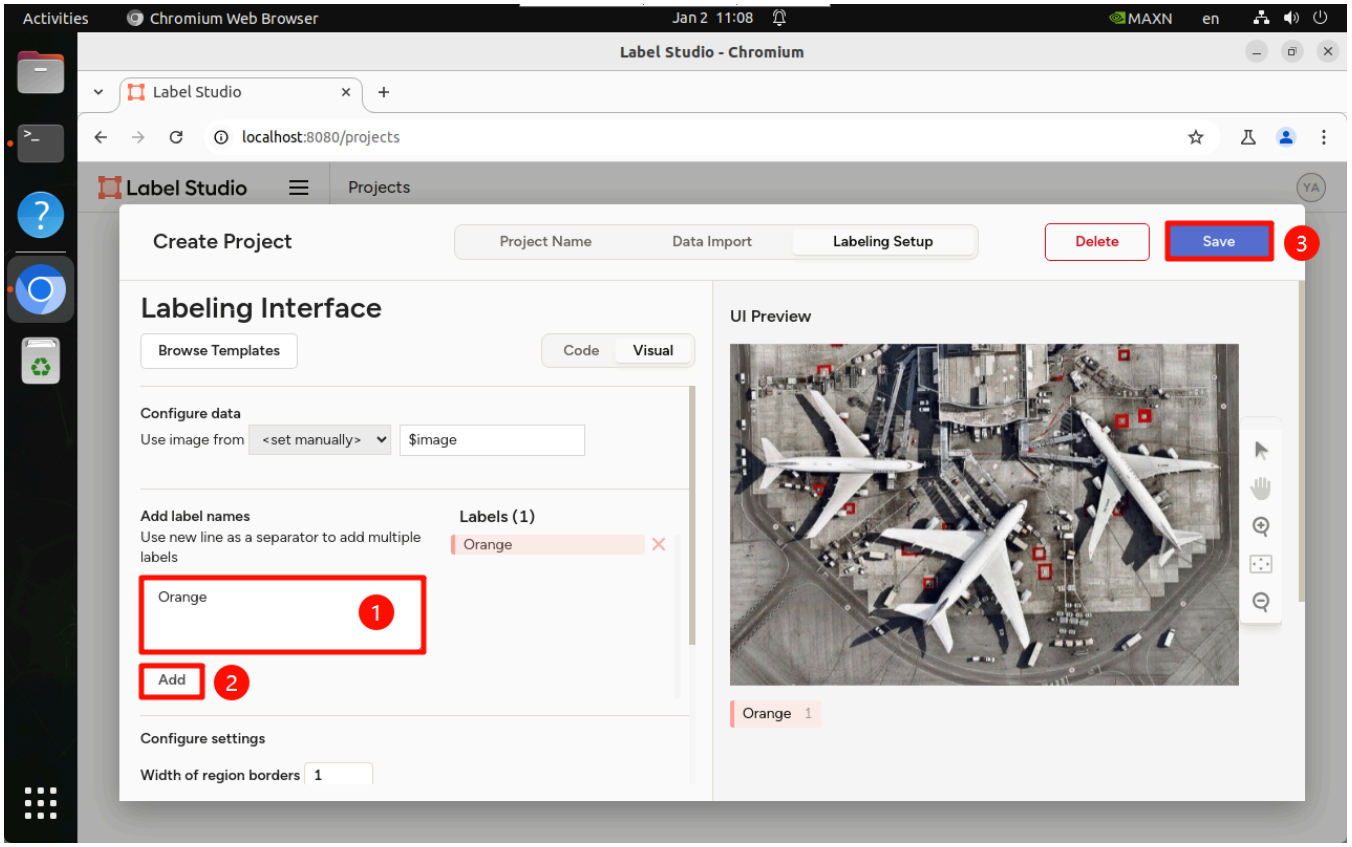
What we demonstrate is the recognition of oranges, so we choose "Object Detection"



Delete the built-in labels:

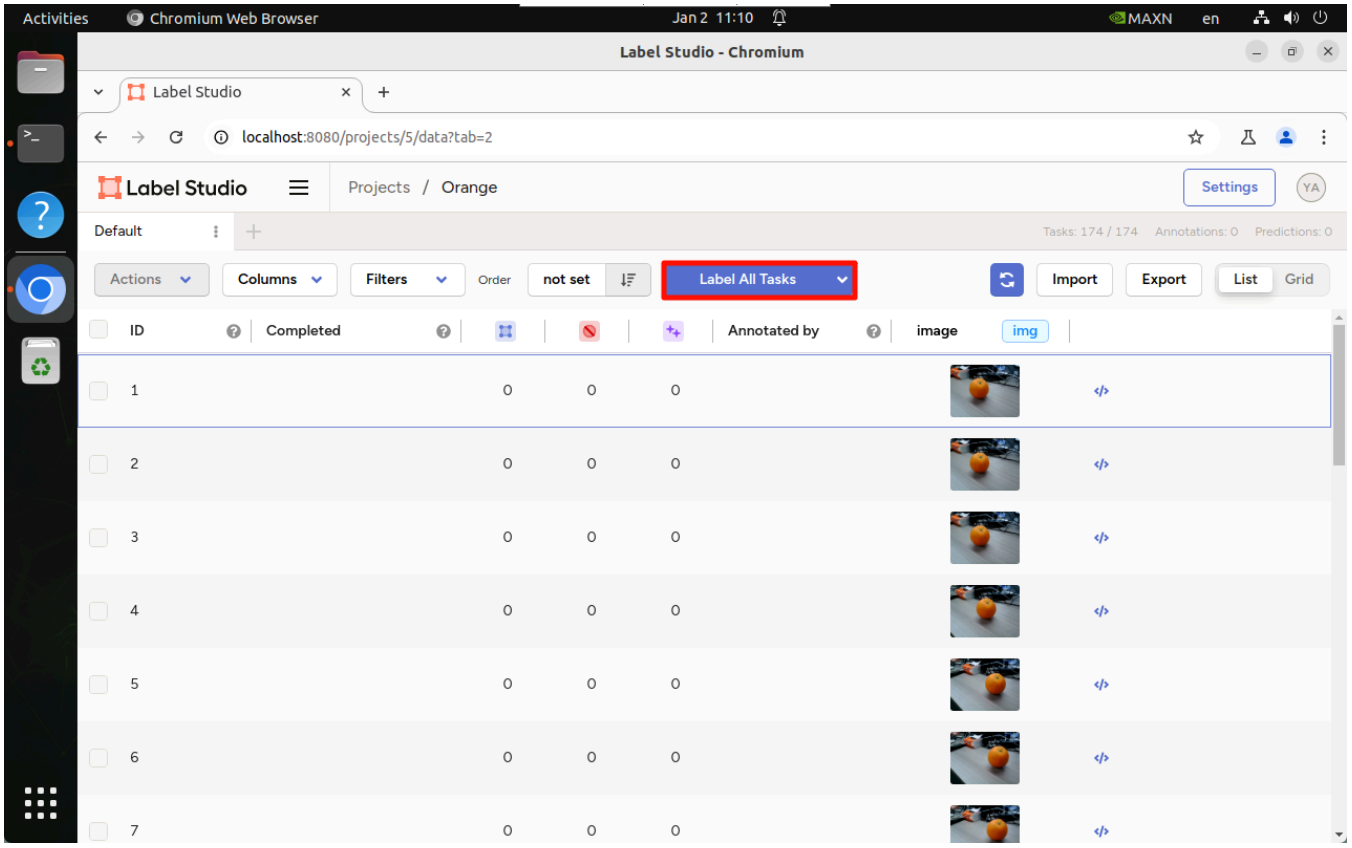


Add the new labels:

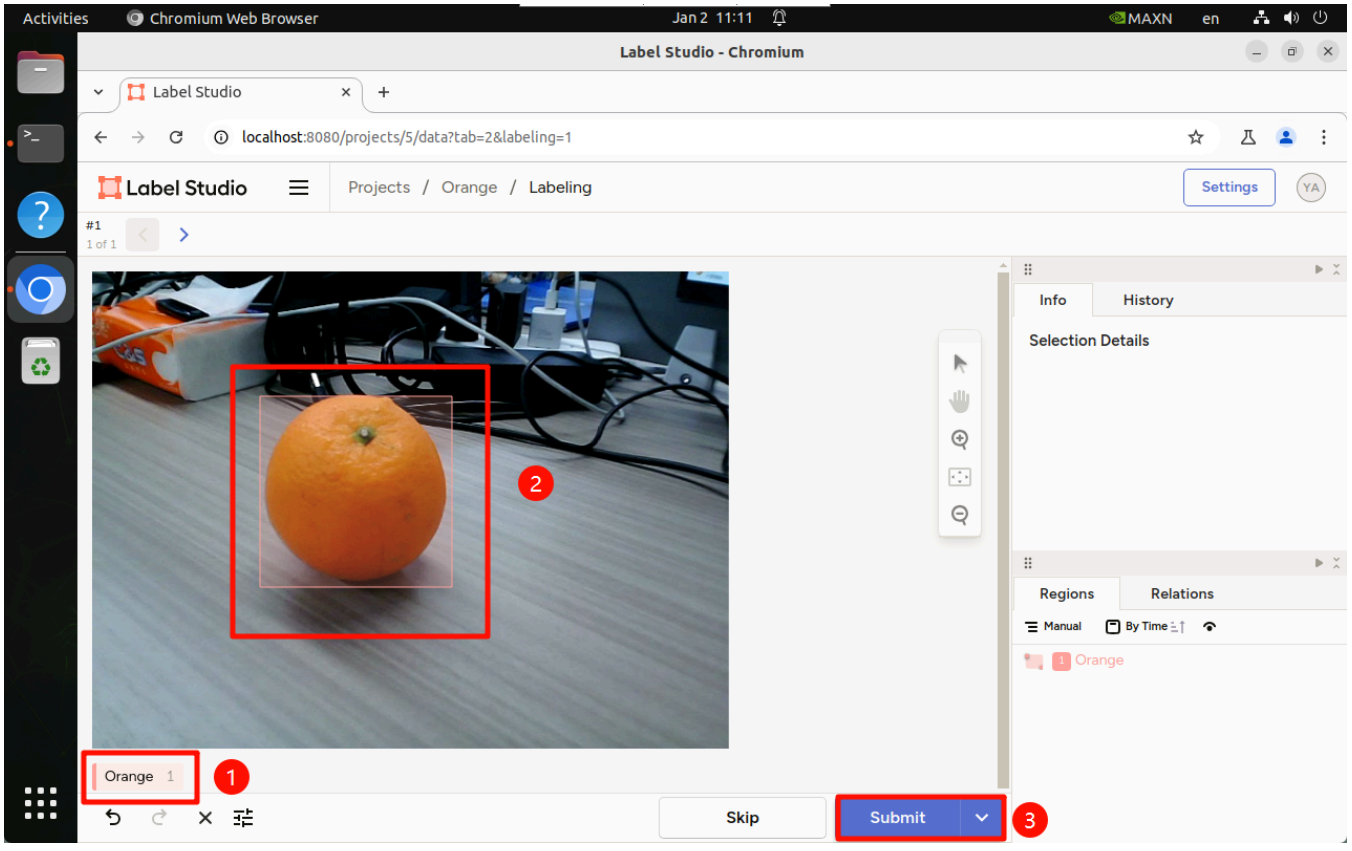


3.5. Dataset annotation

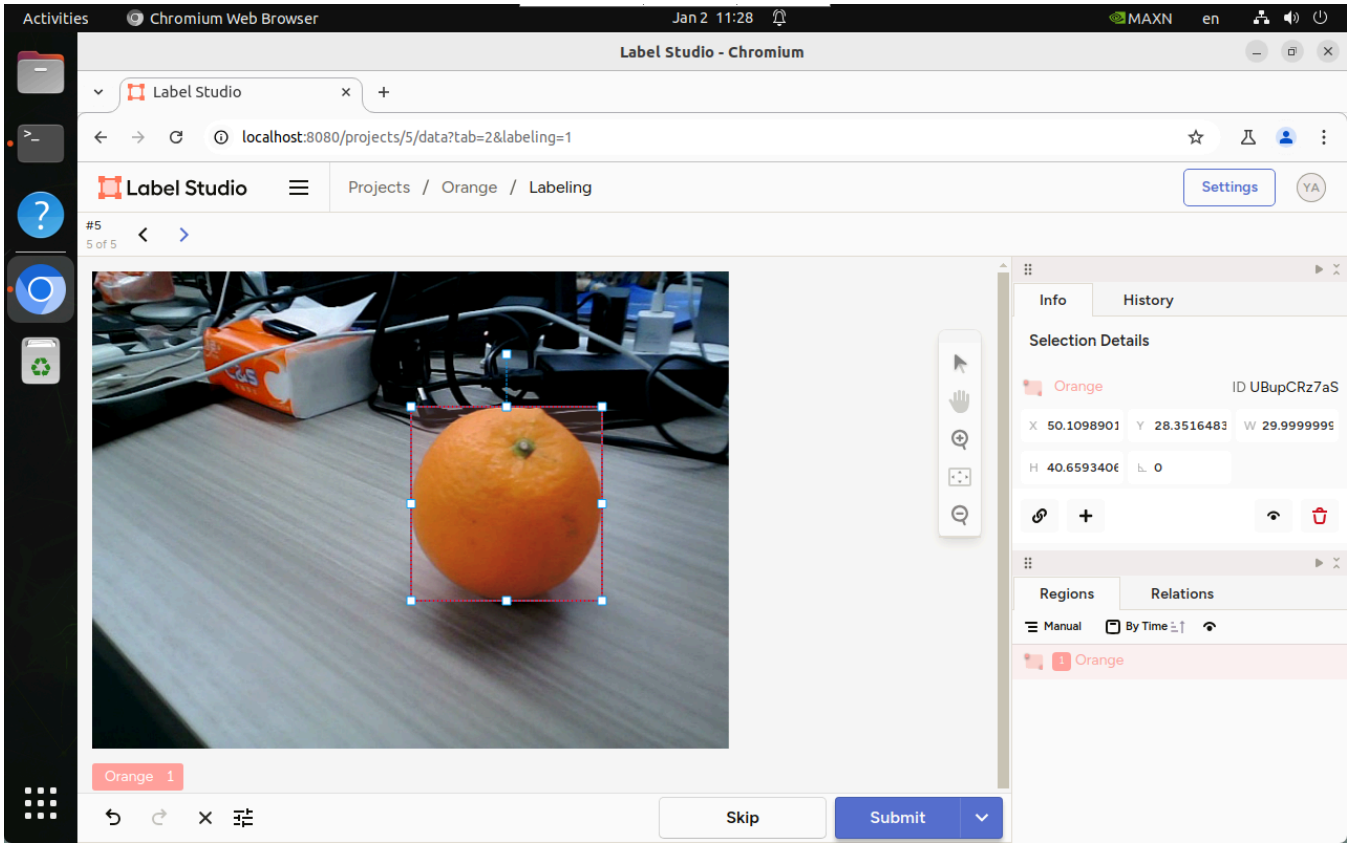
Click to annotate all the tasks:



The annotation steps: first click the label "Orange" and then use the mouse to box the corresponding orange in the image; when it is done, click submit to enter the next annotation

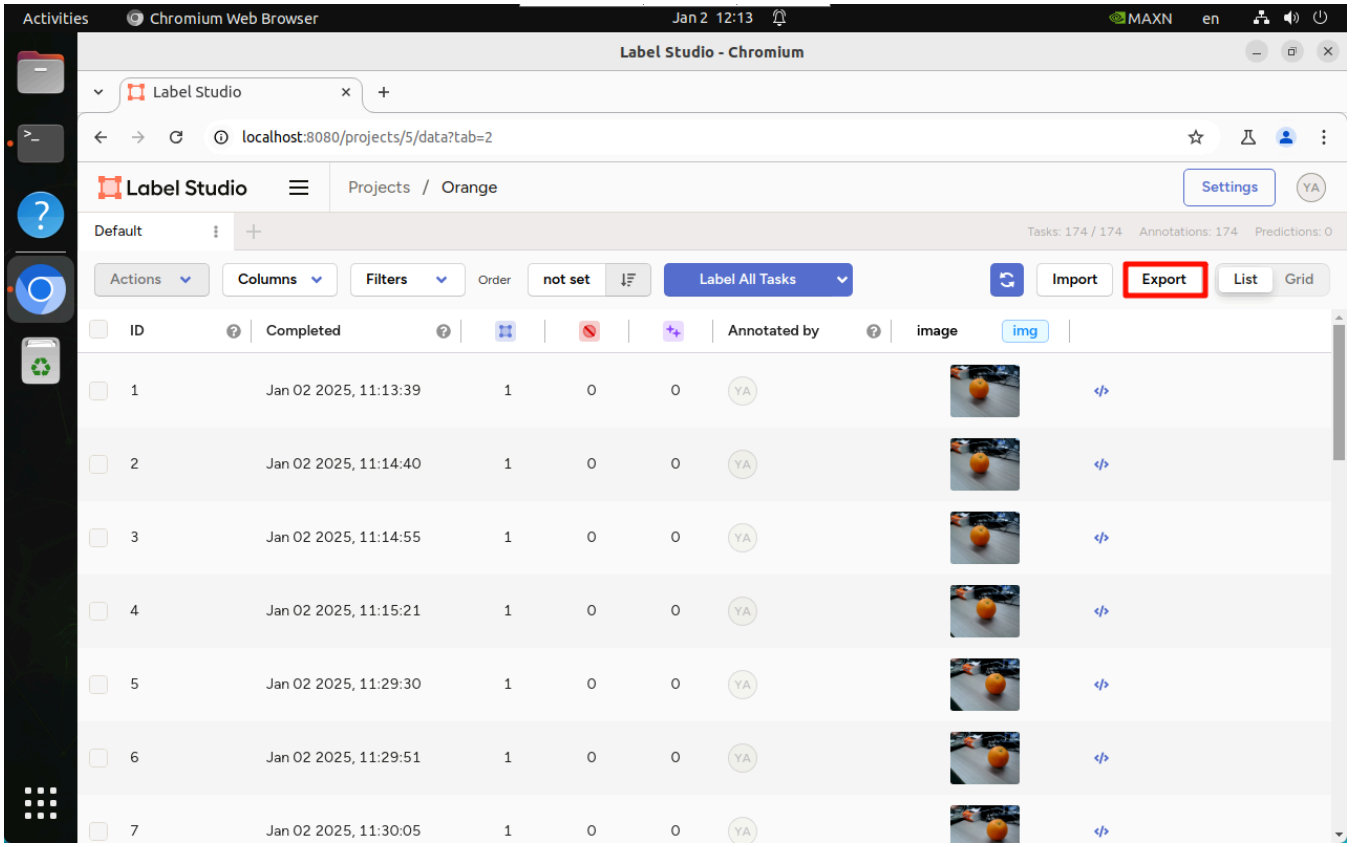


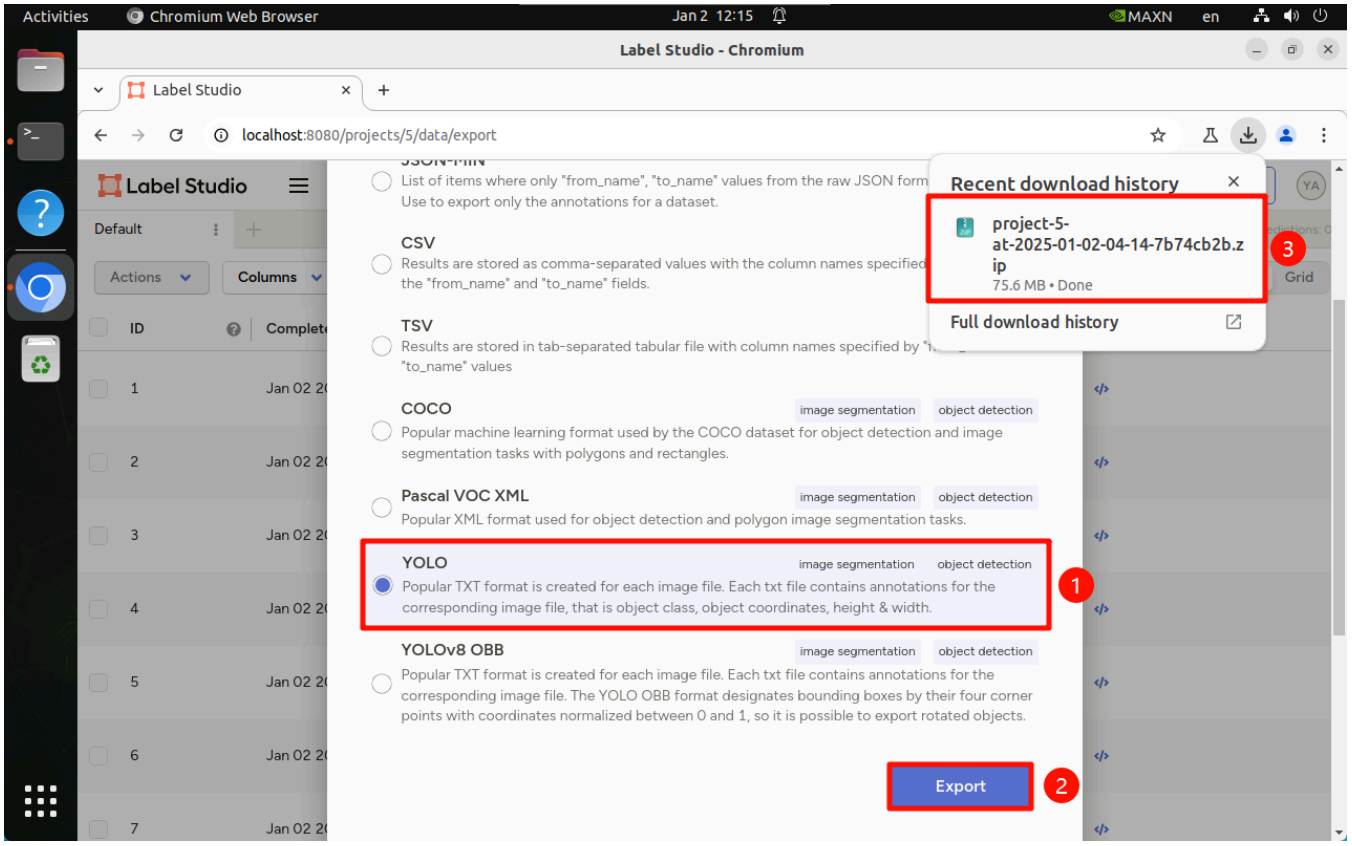
If the boxed object is not a complete orange, you can click the selected box and adjust the position of each edge:



3.6. Exporting the dataset

Choose the YOLO format to export, and the browser automatically downloads an archive





3.7. The directory structure of the dataset

The directory structure corresponding to the archive exported by Label Studio:

```
.
├── images
├── labels
├── classes.txt
└── notes.json
```

We need to split the images and the labels exported by Label Studio into train (the training set) and val (the validation set), and the ratio of the two is about 8:2: the images and the labels inside train and val must correspond to each other by name.

```
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    └── labels
```

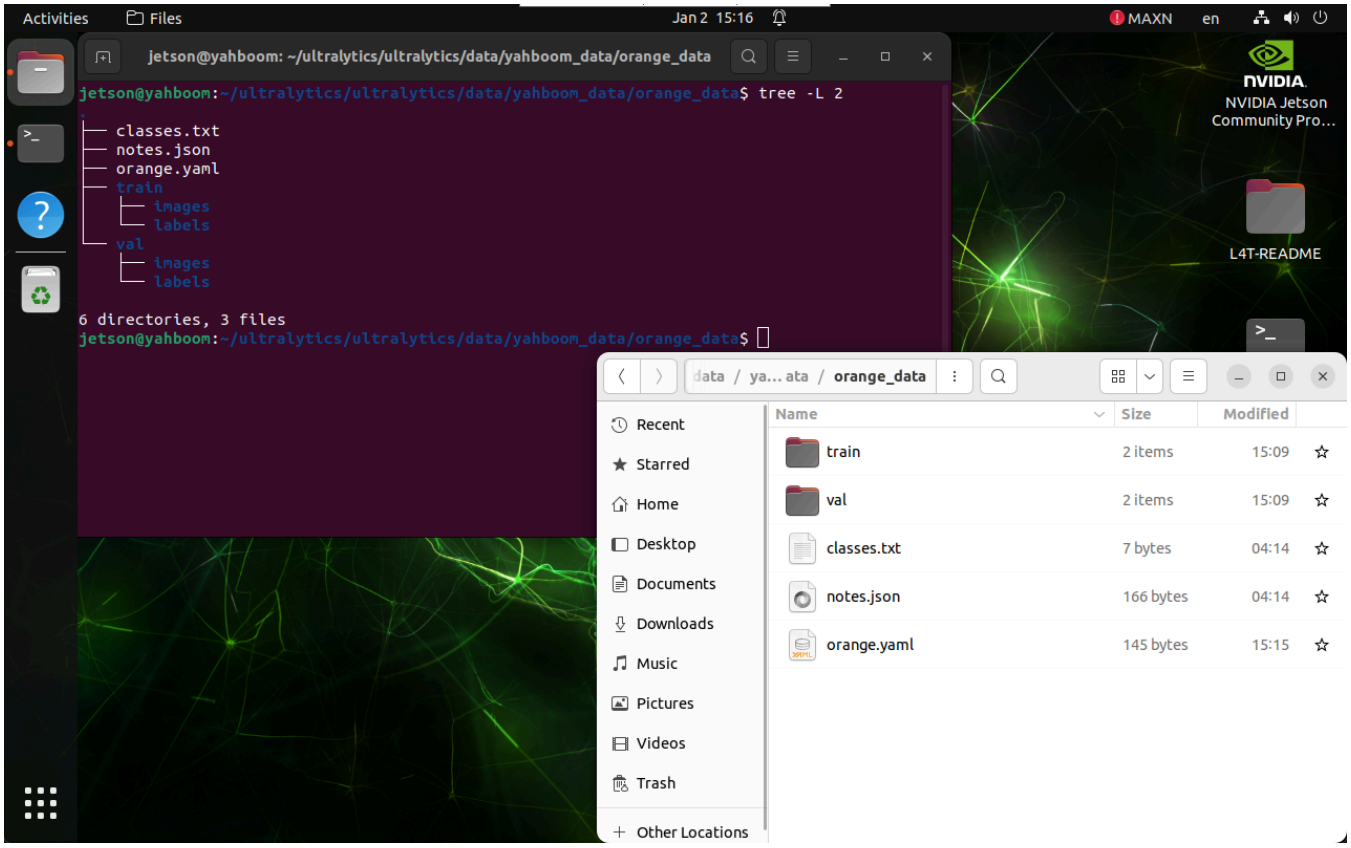
3.8. The dataset configuration file

It contains the paths of the training set and of the validation set, as well as the number and the names of the classes.

```
path: /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data
train: train/images
val: val/images

nc: 1

# Classes
names:
  0: Orange
```



References

https://github.com/HumanSignal/label-studio